Tarea 2: Sistemas Operativos

Torneo de Komikan concurrente

Matías Droguett Juan Pablo Ugaz

Profesor: Yerko Ortiz

3 de Noviembre de 2026

1. Diseño del Esquema de Sincronización

Cada mesa ejecuta trece hilos de fichas (Puma y doce Perros) y un hilo Juez. El estado se divide por responsabilidad: `tablero.c` implementa el monitor `Casillas`, mientras `mesa.c` aplica las reglas y coordina el estado lógico de la partida.

El tablero tiene 28 nodos. Cada nodo es un recurso independiente con `owner[c]`, protegido al consultarlo o modificarlo por su propio `mtx[c]`; `cond[c]` permite esperar hasta que quede libre. Importante: ese mutex se mantiene solo durante la operación breve sobre `owner`; la exclusión del tablero es lógica y persiste mientras la ficha sea su dueña, no consiste en retener un mutex POSIX durante el movimiento. Así, fichas con regiones disjuntas pueden operar simultáneamente. El Perro reserva su origen y vecinos (hasta 5 nodos); el Puma agrega los destinos de salto (hasta 9).

Antes de leer o cambiar `tablero[]`, cada ficha construye y ordena ascendentemente su vecindario, y reserva todos sus nodos. La validación exige que origen, destino y, para una captura, casilla saltada estén reservados. Entonces `juez_validar` consulta el estado autoritativo y `juez_aplicar` actualiza el tablero; la ficha conserva las reservas hasta registrar el movimiento y liberar la región en orden inverso. `vista[]` es una copia informativa: `mtx_mesa` protege la instantánea, estados, contadores y registro, pero no sustituye los monitores que protegen `tablero[]`. La jerarquía es casillas (por índice ascendente), `mtx_mesa` y, al emitir mensajes, consola. El Juez no toma casillas. Los datos se inicializan antes de crear los hilos y se consultan al final después de unirlos; `terminar` y los latidos son atómicos.

2. Análisis de Riesgos de Concurrencia

Deadlock. La condición necesaria de espera circular se elimina imponiendo un orden total: todos reservan nodos en orden ascendente y liberan en orden inverso. Si una ficha espera un nodo, ya posee solo nodos menores; por tanto, las aristas de cualquier cadena de espera avanzan estrictamente por índice y no pueden cerrar un ciclo. Además, `tomar` libera el mutex POSIX antes de esperar en la condición, y si la partida termina durante la reserva, `casillas_bloquear` deshace las reservas parciales. El Juez nunca espera por una casilla mientras mantiene `mtx_mesa`. La ruta de fallo simulado también se recupera: el Juez detecta la falta de latido, une los hilos y libera las reservas huérfanas.

Livelock. La contención no se resuelve con reintentos activos: quien encuentra una casilla ocupada espera en `cond[c]` por tramos de 50 ms, actualiza su latido y vuelve a comprobar el predicado; al liberar una casilla se señaliza a un esperador. Cada ficha prueba un conjunto finito de candidatos y luego libera su región, y sus ciclos incluyen un retardo configurable. Esto evita que el protocolo de reserva consuma CPU girando. No obstante, no existe garantía de progreso de cada movimiento: estrategias sin mejora, conflictos o falta de movimientos pueden impedirlo. El Juez detecta al Puma acorralado y termina por condición de victoria, abandono o duración máxima configurada (empate), evitando una partida indefinida.

Starvation. La señalización y el orden de adquisición no son una política FIFO: POSIX no garantiza qué esperador despierta primero y tampoco hay turnos ni cuota por ficha. Por ello, no se puede afirmar que el código impida formalmente la inanición de una ficha bajo cualquier planificador. La espera por condición, los ámbitos locales de reserva y los latidos actualizados mientras se espera reducen espera activa y falsas detecciones de fallo, pero no constituyen una garantía de equidad. El límite temporal de la partida acota cuánto puede prolongarse el torneo; no garantiza que todos se muevan antes del empate.

3. Evaluación

La estrategia es sencilla de auditar: monitores por nodo evitan un mutex global del tablero, y la validación bajo la región reservada mantiene consistentes las reglas y las capturas. Con tablero fijo de 28 nodos, formar y ordenar el vecindario cuesta $O(V^2)$ por inserción, con $V \leq 9$; la deduplicación y la validación de hasta ocho propuestas también tienen cotas constantes pequeñas. El coste real del movimiento incluye la contención y la salida a log/consola, que puede prolongar la sección de `mtx_mesa`.

La latencia no es de tiempo real: depende del retardo configurado, la planificación y la ocupación de los nodos. Una liberación señaliza al esperador; el tramo de 50 ms sirve para reexaminar la condición, no es una demora obligatoria para

todo movimiento. El Juez reevalúa cada 100 ms o al recibir eventos como captura o Puma acorralado. En una mesa pueden estar activas hasta 13 fichas, con movimientos concurrentes cuando sus vecindarios no se solapan; con M mesas, el límite es $13M$ hilos de fichas, además de Jueces y servidor. Se gana concurrencia frente al bloqueo global, a cambio de más primitivas y de no ofrecer equidad estricta; el mutex de mesa y la impresión detallada son los principales puntos de serialización.